

Angela Wei

Data 260

Homework 2

Section 0:

Hardware Used: HP Pavilion Windows 11 Pro

Local Model Used: Ollama3.2

Commit Hash:

Personal Configuration and Domain

SID4: 1339

PORT_BASE: 8439

PREFIX: s1339

SEED: 1339

VERIFY_SEED: 261339

DOMAIN_ID: 3 – Grocery Supply and Recall Notices

Part 1

CSS Styling for Form:

HTML form before additional CSS styling and formatting applied.

Grocery Supply and Recall Notices

Enter Your Information

Primary Field * :

Secondary Field * :

Email * :

Content Description * :

Select Domain * :

☐ I agree to the Terms and Conditions

CSS styling that changes the appearance of the form. Form should now appear more colorful and formatting should be easier on the eye

```
<style>
  body {
    font-family: Verdana, sans-serif;
    width: 90%;
    max-width: 700px;
    margin: 20px auto;
    padding: 20px;
    background-color: #FFD6A7;
    line-height: 1.6;
  }

  form {
    padding: 60px;
    background-color: #FEF9C2;
    border-radius: 8px;
  }

  textarea{
    height: 100px;
    width: 400px;
  }
```

```
  button {
    background-color: #BB8FCE;
    color: #C0392B;
    padding: 10px;
    margin-top: 30px;
    margin-right: 65px;
  }

  button:hover {
    background-color: #A3E4D7;
  }

  #resultsContainer {
    background-color: #FEF9C2;
    padding: 60px;
    border-radius: 8px;
  }
```

Form with CSS styling applied.

Grocery Supply and Recall Notices

Enter Your Information

Primary Field * :

Secondary Field * :

Email * :

Content Description * :

Write your description here

Select Domain * :

Choose

Submit

☐ I agree to the Terms and Conditions

```
.state {
  display: none;
  padding: 15px
}

.state.active {
  display: block;
}

.state-error {
  background-color: #FEF9C2;
  font-family: Verdana, sans-serif;
}

.state-empty{
  background-color: #FEF9C2;
  font-family: Verdana, sans-serif;
}

.state-success {
  background-color: #FEF9C2;
  font-family: Verdana, sans-serif;
  text-align: left;
}
```

Creating Empty, Loading and Error states for Form:

HTML and CSS to display the different states on the form.

```
.state {
  display: none;
  padding: 15px
}

.state.active {
  display: block;
}

.state-error {
  background-color: #FEF9C2;
  font-family: Verdana, sans-serif;
}

.state-empty{
  background-color: #FEF9C2;
  font-family: Verdana, sans-serif;
}

.state-success {
  background-color: #FEF9C2;
  font-family: Verdana, sans-serif;
  text-align: left;
}
```

```
<div id = "resultsContainer">
  <div id = "emptyState" class = "state state-empty active">
    <p>Please finish filling out the form before submitting</p>
  </div>
  <div id = "loadingState" class = "state">
    <p>Processing your submission</p>
  </div>
  <div id = "errorState" class = "state state-error">
    <p id = "errorMessage">There was an error in processing your submission</p>
  </div>
  <div id = "successState" class = "state state-success">
    <p>Sucessfully Submitted!</p>
    <p>Here are your details of your submission:</p>
    <p id = "show_output"></p>
  </div>
</div>
```

A description of less than 25 characters will trigger an error state and subsequent message.

```
// Arrow function to verify there are more than 25 characters
// This should trigger an error state
const validateFormSubmission = () => {
  const confirmInput = document.getElementById('content');
  const value = confirmInput ? confirmInput.value.trim() : '';
  if (value.length <= 25){
    alert('Must be longer than 25 characters');

    // other states are set to inactive when error state becomes active
    document.getElementById('emptyState').classList.remove('active');
    document.getElementById('loadingState').classList.remove('active');
    document.getElementById('successState').classList.remove('active');
    const errorState = document.getElementById('errorState');

    // error state then will become active
    // error message is displayed
    document.getElementById('errorMessage').textContent = 'Must be longer than 25 characters'
    errorState.classList.add('active');

    return false
  }
  return true
};
```

Not accepting the Terms and Conditions will trigger an error state and subsequent message.

```
// Arrow function to check if the terms and conditions are validated
// This should trigger an error state
const validateTermsandConditions = () => {
  const checkTerms = document.getElementById('acceptTerms');
  const confirmCheck = checkTerms ? checkTerms.checked : false;
  if (!confirmCheck){
    alert('Please accept Terms and Conditions');

    // other states are set to inactive when error state becomes active
    document.getElementById('emptyState').classList.remove('active');
    document.getElementById('loadingState').classList.remove('active');
    document.getElementById('successState').classList.remove('active');
    const errorState = document.getElementById('errorState');

    // error state then will become active
    // error message is displayed
    document.getElementById('errorMessage').textContent = 'Please accept Terms and Conditions'
    errorState.classList.add('active');

    return false
  }
  return true
};
```

Submitting the form will trigger the loading state.

```
// try and except for loading state to check if submission is successful or not
try {
  // go into loading state after submitting form
  document.getElementById('emptyState').classList.remove('active');
  document.getElementById('errorState').classList.remove('active');
  document.getElementById('successState').classList.remove('active');
  document.getElementById('loadingState').classList.add('active');

  const getForm_Feedback = {
    primary : document.getElementById("sid4").value,
    secondary : document.getElementById("portBase").value,
    email : document.getElementById("prefix").value,
    content : document.getElementById("content").value,
    domain_info : document.getElementById("domain_id").value,
  };

  // cover the submitted information into JSON format
  const jsonOutput = JSON.stringify(getForm_Feedback);
  console.log("in JSON: ", jsonOutput);
}
```

Error state will be triggered again if the submission is not successful.

This error state is treated like an exception being raised.

```
// go into error state if submission is not successful
// error state is treated like an exception being raised
catch(error){
    document.getElementById('emptyState').classList.remove('active');
    document.getElementById('loadingState').classList.remove('active');
    document.getElementById('successState').classList.remove('active');
    document.getElementById('errorState').classList.add('active');
}
```

Information submitted through the form will be displayed after submission

```
// display details of the the form submission by connecting to HTML
document.getElementById('show_Output').innerHTML = `<strong>Primary Field:</strong> ${primary}
<br> <strong>Email:</strong> ${email}<br>
<strong>Domain:</strong> ${parse_update.domain_info}<br>
<strong>Submitted Date</strong> ${parse_update.submissionDate}<br>
<strong>Total Submissions:</strong> ${form_count}<br>`;
```

Success state is triggered if form submission goes through

```
// go into success state if submission is successful
document.getElementById('loadingState').classList.remove('active');
document.getElementById('successState').classList.add('active');
```

Part 2

FastAPI is imported at the beginning of the file

```
from fastapi import FastAPI, HTTPException
from fastapi.staticfiles import StaticFiles
from fastapi.responses import FileResponse
from pydantic import BaseModel
from typing import List
import uvicorn

app = FastAPI(title="User Management API", version="1.0.0")

app.mount("/static", StaticFiles(directory="static"), name="static")
```

Webpage will launch on designated PORT_BASE when main.py starts running

```
if __name__ == "__main__":
    #Open the browser automatically upon running file
    #use port 8439
    #is assigned PORT_BASE
    webbrowser.open("http://localhost:8439")
    uvicorn.run(app, host="0.0.0.0", port=8439)
```

If a user does not exist yet, a new one is created.

```
#Create a new user record
@app.post("/api/users", response_model=User, status_code=201)
async def create_user(user_data: UserCreate):
    #if there is an empty string or blank, then reject it
    #raise HTTP 400 error if this happens
    if not user_data.primary_field.strip():
        raise HTTPException(status_code=400, detail="Please enter a primary field")

    #look for current highest ID and then add 1
    new_id = max([user.id for user in users], default=0) + 1

    #create new user object
    new_user = User(id=new_id, primary_field=user_data.primary_field, secondary_field=user_data.secondary_field)

    #add the new user object to list
    users.append(new_user)

    #return new user info
    print(f"A new record has been created: {new_user}")
    return new_user
```

Existing user records can be updated.

```
#Update a record
@app.put("/api/users/{user_id}", response_model=User)
async def update_user(user_id: int, user_data: UserUpdate):
    #search for user by their ID
    user = next((user for user in users if user.id == user_id), None)

    #raise an HTTP 404 if the user is not found
    if not user:
        raise HTTPException(status_code=404, detail="User not found")

    user.primary_field = user_data.primary_field
    user.secondary_field = user_data.secondary_field
    print(f"Updated record fields: {user}")
    return user
```

Process to delete an existing user by selecting the one with the highest id number.

```
#Delete a record with the highest ID
@app.delete("/api/users/delete-highest", response_model = User)
async def delete_highest_user(user_id: int):
    #search for user by their ID
    #identify ID by the current highest number
    global users
    user_index = next((index for index, user in enumerate(users) if user.id == user_id), None)

    #raise an HTTP 404 if the user is not found
    if user_index is None:
        raise HTTPException(status_code=404, detail="Records are not found. Cannot be deleted.")

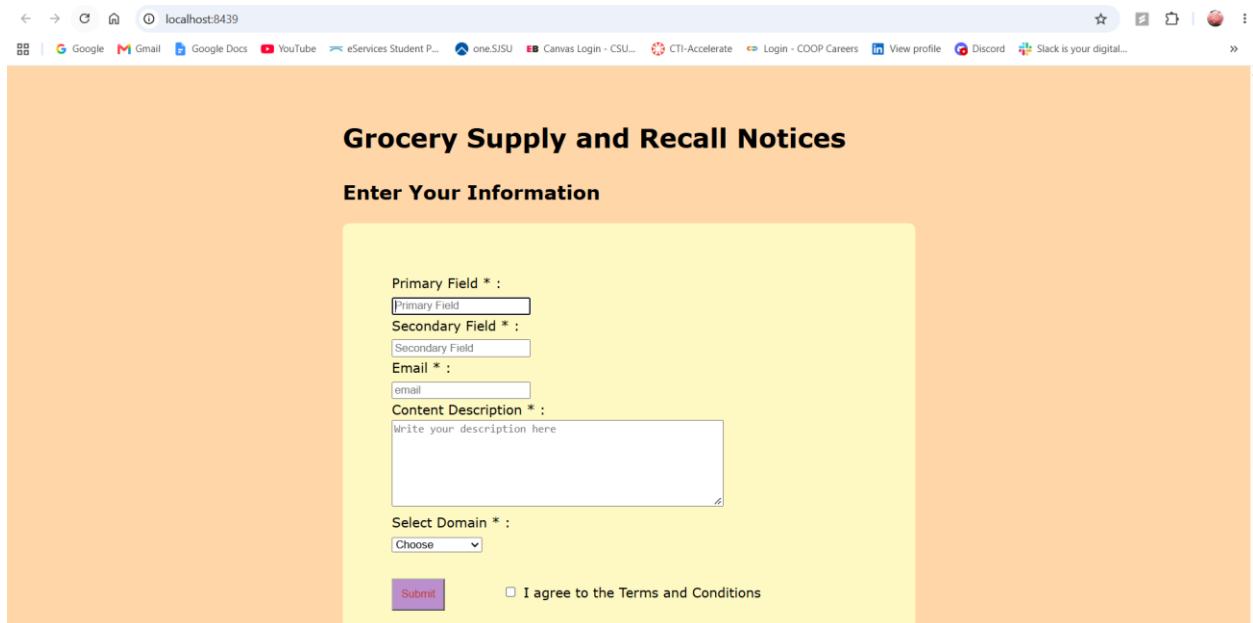
    #iterate through all available users
    #remove the user if found
    highest_id = users[0]
    for user in users:
        if user.id > highest_id.id:
            highest_id = user

    users.remove(highest_id)
    print(f"Record with the highest ID has been deleted: {highest_id}")
    return highest_id
```

Application running in terminal. Typing in command will automatically start the page in localhost in the designated PORT_BASE

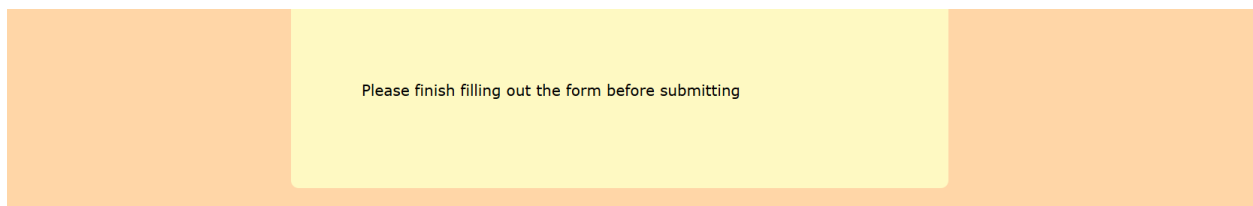
```
PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code> python main.py
INFO: Started server process [35148]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8439 (Press CTRL+C to quit)
INFO: 127.0.0.1:57781 - "GET / HTTP/1.1" 200 OK
INFO: 127.0.0.1:57781 - "GET /HW1%20-%20Angela%20Wei.js HTTP/1.1" 404 Not Found
INFO: 127.0.0.1:57781 - "GET /favicon.ico HTTP/1.1" 404 Not Found
```


Webpage showing up in designated PORT_BASE.



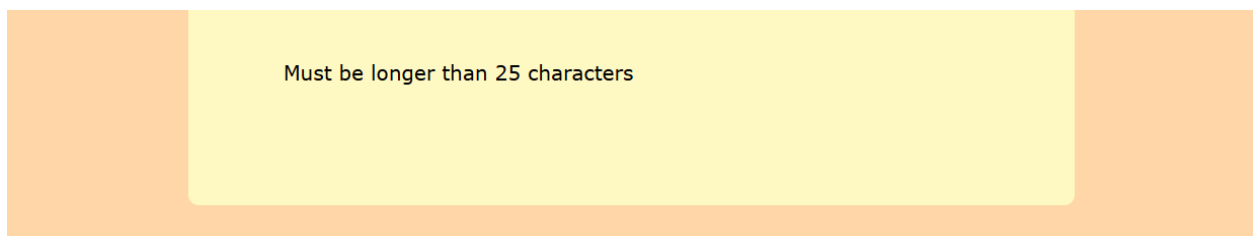
The screenshot shows a web browser window with the address bar displaying 'localhost:8439'. The browser's tab bar includes links to Google, Gmail, Google Docs, YouTube, eServices Student P..., one.SJSU, Canvas Login - CSU..., CTI-Accelerate, Login - COOP Careers, View profile, Discord, and Slack is your digital... The main content area has an orange background and features a form titled 'Grocery Supply and Recall Notices' with the subtitle 'Enter Your Information'. The form is contained within a yellow box and includes the following fields: 'Primary Field *' with a text input, 'Secondary Field *' with a text input, 'Email *' with a text input, 'Content Description *' with a text area, and 'Select Domain *' with a dropdown menu. At the bottom of the form is a purple 'Submit' button and a checkbox labeled 'I agree to the Terms and Conditions'.

Empty state before filling out form.



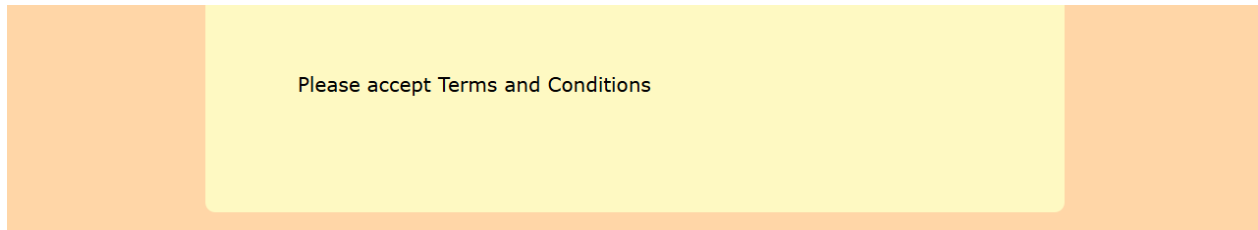
The screenshot shows a yellow rectangular box centered on an orange background. Inside the box, the text 'Please finish filling out the form before submitting' is displayed in a dark gray font.

Error state if description is less than 25 characters



The screenshot shows a yellow rectangular box centered on an orange background. Inside the box, the text 'Must be longer than 25 characters' is displayed in a dark gray font.

Error state if user does not accept Terms and Conditions



User inputs being displayed after form goes into success state



Part 3

Import TypedDict from typing for use in the AgentState class.

```
from typing import List, Dict, Any, Iterable, Tuple, TypedDict
```

Setting up Agent State Node as a class

```
#Agent State Node
class AgentState(TypedDict):
    title: str
    content: str
    email: str
    strict: bool
    task: str
    llm: Any
    transcript: Annotated[List[Dict[str, Any]], operator.add]
    planner_proposal: Dict[str, Any]
    reviewer_feedback: Dict[str, Any]
    final_output: Dict[str, Any]
    turn_count: int
    max_turns: int
```

Calling the Agent State Node in the main function

```
task = "Extract 3 topical tags and a summary of at most 25 words from provided title and content"
state = AgentState = {
    "title": args.title,
    "content": args.content,
    "email": args.email,
    "strict": args.strict,
    "task": task,
    "llm": llm,
    "transcript": [],
    "planner_proposal": {},
    "reviewer_feedback": {},
    "final_output": {},
    "turn_count": 0,
    "max_turns": args.turns
}
```

Planner agent as its own function

```
#Planner Node carrying Planner Agent
def Planner_Node(state: AgentState):
    planner = SimpleAgent(name = "Planner",
                           system = ("Generate a draft with: "
                                     "3 topical tags based on the given topic."
                                     "A summary with 25 words maximum. No more than 25 words."
                                     "tags format is [tag1, tag2, tag3]"
                                    ),
                           model = state["llm"])

    t0 = time.time()
    suggestions = planner.respond(state["transcript"], state["task"], state["title"], state["content"], state["strict"])
    t1 = time.time()

    print(f"\n--- Planner ({int((t1 - t0) * 1000)} ms) ---\n{json.dumps(suggestions, indent=2)}")

    return {"planner_proposal": suggestions, "transcript": [{"role": "Planner", "content": json.dumps(suggestions)}]}
```

Reviewer agent as its own function

```
#Reviewer Node carrying Reviewer Agent
def Reviewer_Node(state: AgentState):
    reviewer = SimpleAgent(name = "Reviewer",
                           system = ("Review everything from the Planner's output"
                                     "Planner should have generate 3 topical tags"
                                     "The summary should have 25 words maximum. No more than 25 words"
                                     "tags format is [tag1, tag2, tag3]"
                                    ),
                           model = state["llm"])

    t0 = time.time()
    feedback = reviewer.respond(state["transcript"], state["task"], state["title"], state["content"], state["strict"])
    t1 = time.time()

    print(f"\n--- Reviewer ({int((t1 - t0) * 1000)} ms) ---\n{json.dumps(feedback, indent=2)}")

    return {"reviewer_feedback": feedback, "transcript": [{"role": "Planner", "content": json.dumps(feedback)}]}
```

Supervisor node to count and track and change the current state

```
#set up Supervisor Node
def supervisor_node(state: AgentState):
    #modify the state
    #counter increments
    turn_count = state["turn_count"] + 1

    print(f"Turn Count: {turn_count}")

    return {"turn_count": turn_count}
```

Routing function to make decision on whether to go to planner or reviewer based on current state.

```
#Routing
def router_logic(state: AgentState):
    #check if any proposals exist
    proposal_exists = bool(state.get("planner_proposal"))

    #look for feedback from reviewer
    reviewer_feedback = state.get("reviewer_feedback") or {}

    #check for issues in the reviewer feedback
    issues_exist = reviewer_feedback.get("has_issues", False)

    #continue to go to planner no proposals are seen
    if not proposal_exists:
        print("No proposal. Going to Planner")
        return "planner"
    #go to reviewer if no proposals but issues are found by reviewer
    if issues_exist and state["turn_count"] < state["max_turns"]:
        print("Proposal made. Going to Reviewer")
        return "reviewer"

    print("Process Complete")
    return END
```

Import StateGraph from langgraph to run stream method.

```
from langgraph.graph import StateGraph, START, END
```

Everything connected in the main function.

```
build = StateGraph(AgentState)

build.add_node("supervisor", supervisor_node)
build.add_node("planner", Planner_Node)
build.add_node("reviewer", Reviewer_Node)

build.add_edge(START, "supervisor")
build.add_conditional_edges("supervisor", router_logic, {"planner": "planner", END: END})

build.add_edge("planner", "reviewer")
build.add_edge("reviewer", "supervisor")

graph = build.compile()

for event in graph.stream(initial_state):
    for node_name, node_output in event.items():
        print(f"Stream update from {node_name}")
```

Output from run:

```
PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code> python agents_demo.py --model llama3.2
Turn Count: 1
No proposal. Going to Planner
Stream update from supervisor

--- Planner (15803 ms) ---
{
  "Grocery Supply and Recall Notices": [
    "recall",
    "grocery",
    "contamination"
  ],
  "Summary: 25 words": "Groceries recalled due to contamination, foreign objects, or allergens. Products removed from stores to ensure consumer safety and prevent health risks."
}
Stream update from planner

--- Reviewer (3584 ms) ---
{
  "recall": [
    "recall",
    "grocery",
    "contamination"
  ],
  "Summary": "Groceries recalled due to contamination, foreign objects, or allergens. Products removed from stores."
}
Stream update from reviewer
Turn Count: 2
Process Complete
Stream update from supervisor

Finalized Output
{
  "tags": [
    "Recall",
    "Grocery",
    "Safety"
  ],
  "Summary": "Grocery recall notices issued for contaminated products with foreign objects or allergens, products removed from store."
}
PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code> |
```

Altering the reviewer node so it will always return issues.

```
#Reviewer Node carrying Reviewer Agent
def Reviewer_Node(state: AgentState):
    reviewer = SimpleAgent(name = "Reviewer",
                            system = [
                                "Review everything from the Planner's output"
                                "Planner should have generate 3 topical tags"
                                "The summary should have 25 words maximum. No more than 25 words"
                                "tags format is [tag1, tag2, tag3]"
                                "Always flag any issues regardless of what is in the input"
                                "The JSON output keys 'has_issues' is true. Any issues and failure should be explained in the 'feedback'"
                            ],
                            model = state["llm"])

    t0 = time.time()
    feedback = reviewer.respond(state["transcript"], state["task"], state["title"], state["content"], state["strict"])
    t1 = time.time()

    print(f"\n--- Reviewer ({int((t1 - t0) * 1000)} ms) ---\n{json.dumps(feedback, indent=2)}")

    return {"reviewer_feedback": feedback, "transcript":[{"role": "Planner", "content":json.dumps(feedback)}]}
```

Output showing issues detected in the process.

```
PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code> python agents_demo.py --model llama3.2
Turn Count: 1
No proposal. Going to Planner
Stream update from supervisor

--- Planner (14591 ms) ---
{
  "Grocery Supply and Recall Notices": [
    "recall",
    "grocery",
    "contamination"
  ],
  "Summary: 25 words": "Groceries recalled due to contamination, foreign objects, or allergens. Products removed from stores to ensure consumer safety and prevent health risks."
}
Stream update from planner

--- Reviewer (5741 ms) ---
{
  "has_issues": true,
  "feedback": "Issues detected: missing 'content' key in output, missing 'transcript' key in output. Summary too long. Tags not extracted correctly.",
  "tags": [
    "recall",
    "grocery",
    "contamination"
  ],
  "summary": "Groceries recalled due to contamination or allergens. Products removed for consumer safety."
}
Stream update from reviewer
Turn Count: 2
Process Complete
Stream update from supervisor

Finalized Output
{
  "tags": [
    "Recall",
    "Grocery",
    "Safety"
  ],
  "summary": "Grocery recall notices issued for contaminated products with foreign objects or allergens, products removed from store."
}
```

Part 4

Fixed Domain input file to test:

```
{
  "title": "Grocery Supply and Recall Notices",
  "content": "Groceries will be recalled if products are contaminated, contain foreign objects or contain allergens. Products are then removed from store."
}
```

Input in the file:

```
.ArgumentParser()
nt("--title", default="Grocery Supply and Recall Notices")
nt("--content", default="Groceries will be recalled if products are contaminated, contain foreign objects or contain allergens. Products are then removed from store.")
```

GitHub:

<https://github.com/Aywei2001/data260--SID4->